

PulseBuild

Source of Truth v2.0

Complete Product, Reactive Agent, UX, Design, Architecture, Data, and Cursor Execution Specification

Document Rule	Meaning
Primary use	This is the reference Cursor must read before planning or implementation. It replaces the shallow engineering supplement and becomes the controlling specification for the MVP.
Authority	If a prompt conflicts with this document, Cursor must stop and ask the Product Owner before coding.
Scope	Defines what PulseBuild is, how the reactive agent behaves, what the command center shows, what data exists, and how Cursor must build it.
Coding gate	This document allows planning and scaffolding only. Actual implementation should follow Product Owner phase approval.

Product promise: One site. One live reflex layer. No hidden blockers.

Table of Contents

- 1. Executive Summary
- 2. Research and Problem Foundation
- 3. Product Mission and Non-Negotiable Principles
- 4. Business Goals and Success Metrics
- 5. Target Users, Roles, and Permissions
- 6. MVP Scope, Out of Scope, and Build Roadmap
- 7. Construction Operating Model
- 8. Reactive Agent Brain
- 9. Signal Taxonomy and Priority Matrix
- 10. Dependency, Drift, and Health Scoring
- 11. Detailed Screen Specifications
- 12. Feature Specifications
- 13. Chat and Voice Agent Specification
- 14. Command Center Live Interaction Model
- 15. Design System and Visual Identity
- 16. Component Library
- 17. Technical Architecture
- 18. Conceptual Database Schema
- 19. Simulation Engine
- 20. Cursor Rules and Development Workflow
- 21. Acceptance Criteria, Testing, and Definition of Done
- 22. Decision Log Seed
- 23. Cursor Phase 1 Completion Prompt

1. Executive Summary

PulseBuild is a reactive construction operations agent designed for active job sites where small disruptions become expensive delays. It continuously watches simulated construction signals such as material delays, inspection failures, crew shortages, equipment conflicts, weather risks, permit delays, and task status changes. For each signal, PulseBuild classifies priority, identifies dependencies, predicts cascading impact, recommends corrective action, routes human approval when needed, and verifies whether the response solved the problem.

PulseBuild is not a generic chatbot, not a scheduling dashboard, and not a construction ERP. It is a reflex layer for construction operations. The product is built around one operating question: **What is preventing productive work right now?**

What PulseBuild must do

- Show the highest priority operational blocker within five seconds of opening the app.
- Ingest live simulated signals and update the interface automatically without manual refresh.
- Explain why a signal matters by showing priority, evidence, downstream tasks, and expected impact.
- Recommend one practical next action for every Tier 1 and Tier 2 signal.
- Require approval before high-impact operational actions such as crew reassignment or inspection rescheduling.
- Verify whether the action improved the blocked condition and log the outcome.
- Feel like a professional construction command center, not a static dashboard.

2. Research and Problem Foundation

Construction operations are coordination-heavy. A superintendent may coordinate crews, subcontractors, inspections, material deliveries, permits, equipment, weather exposure, and daily production targets at the same time. The pain is not only that disruptions happen. The real pain is that disruptions are discovered late, their downstream effects are misunderstood, and reactions are often based on memory instead of a shared operational system.

Construction Pain	Current Reality	PulseBuild Product Response
Idle labor	Crews wait because materials, inspections, or equipment are not ready.	Detect idle risk and recommend alternate work before labor is wasted.
Cascading delays	One blocked task pushes other dependent tasks but the chain is not visible in real time.	Walk dependencies and show affected tasks, crews, and milestones.
Inspection blockers	Work cannot continue legally or safely when required inspection status is missing or failed.	Flag as Tier 1, block dependent work, and recommend approved follow-up.
Weather disruption	Outdoor tasks may be scheduled even when weather changes.	Compare weather events to task type and recommend resequencing.
Communication overload	Updates spread across calls, texts, whiteboards, and spreadsheets.	Create one live timeline with signal analysis, recommendation, approval, and verification.
Unclear accountability	Teams forget who reacted, what was approved, and whether it worked.	Write every signal and reaction to an audit log.

Approved Insight

Construction does not only need better planning. It needs faster operational reflexes. PulseBuild exists because plans fail in the field, and the site needs a system that detects disruption, reasons through impact, and drives the next operational response.

3. Product Mission and Non-Negotiable Principles

Principle	Definition
Mission	Help construction teams detect, understand, and respond to operational blockers before productive work is lost.
Core product principle	One site. One live reflex layer. No hidden blockers.
Agent boundary	PulseBuild recommends and explains operational actions. It does not override safety protocols, contractual obligations, or superintendent authority.
Design goal	Make job-site disruption feel visible, prioritized, and actionable through a futuristic but practical command center.
Engineering goal	Build a deterministic-first, agent-assisted application with clear domain logic, reusable components, and seeded MVP data.
Cursor goal	Make every feature traceable to this document so Cursor does not invent scope or behavior.

Non-Negotiable Product Rules

- Every signal must belong to one project and one project timeline.
- Every Tier 1 and Tier 2 signal must generate a recommendation.
- Every recommendation must show evidence and downstream impact.
- Every high-impact action must require human approval.
- Every approved or rejected action must be logged.
- Every executed action must have a verification attempt.
- The agent must never invent project facts outside seeded data or approved user input.
- The Product Owner approves major product, design, architecture, and scope decisions.

4. Business Goals and Success Metrics

The immediate goal is to ship a convincing MVP that proves the reactive loop. The MVP does not need real sensors or enterprise integrations. It must prove that an operational event can enter the system, trigger analysis, generate a recommendation, and close the loop through verification.

Goal	Signal	Target for MVP
Fast triage	Signal classified immediately after ingestion	Priority visible in under 1 second after simulated event.
Cascade awareness	Downstream dependencies identified	At least 90% of seeded dependency chains shown correctly.
Actionable recommendations	Recommendation produced for each critical disruption	100% of Tier 1 and Tier 2 signals produce a concrete next action.
Human control	Risky actions require approval	Crew/schedule/inspection/vendor actions cannot execute without approval.
Closed-loop behavior	Verification outcome logged	At least one full detect - react - verify loop demonstrated.
Polished experience	User understands status quickly	Command Center answers top blocker in under 5 seconds.
Cursor readiness	Implementation follows spec	No feature added unless mapped to this document or approved separately.

5. Target Users, Roles, and Permissions

PulseBuild is designed for field operations users who need fast clarity under pressure. The interface must be simple enough to use during a busy job-site morning, but structured enough that a project manager can trust the audit trail later.

Role	Primary Need	PulseBuild Usage
Superintendent	Know what is blocking productive work and what to do first.	Command Center, approvals, top blocker, recovery plan, live timeline.
Project Manager	Understand schedule risk and accountability.	Health Score, Drift Score, reaction history, unresolved issues, reports.
Foreman	Report field conditions and receive action updates.	Signal input, crew status, assigned recovery actions.
Coordinator	Track deliveries, inspections, permits, and logistics.	Material events, inspection status, permit updates, notifications.
Safety Lead	Ensure safety-related events are never handled automatically.	Safety alerts, approval guardrails, audit trail.
Demo User	Experience the full reactive loop in a classroom MVP.	Start simulation, observe live updates, chat, approve, verify.

Permission Rules for MVP

- MVP may use a single demo user role but the data model must support future role expansion.
- Superintendent can approve recommendations.
- Project Manager can review but not necessarily execute field actions in later versions.
- Safety-related signals always require human review regardless of role.
- Cursor must not build authentication until approved; use mock roles for prototype behavior.

6. MVP Scope, Out of Scope, and Build Roadmap

In Scope for MVP	Out of Scope for MVP
Single simulated construction project	Live integration with Procore, Autodesk, Oracle, or SAP
Seeded project data	Real vendor APIs or automated vendor messages
Live signal simulation	Production authentication and enterprise permissions
Reactive Agent loop	Full BIM or 3D model integration
Priority tiers and dependency analysis	Real sensor hardware integration
Health Score and Drift Score	Contract/legal/payment decisions
Recommendation queue	Automated safety decisions
Approval and verification workflow	Full multi-project portfolio management
Chat assistant and optional voice announcements	Unrestricted general AI assistant

Build Roadmap

Priority	Milestone	Purpose
1	Foundation and design system	Create React app shell, theme tokens, layout grid, and reusable components.
2	Seeded data model	Add static project, tasks, crews, materials, inspections, weather, and signals.
3	Reactive engine	Classify signals, calculate priority, walk dependencies, and update health/drift.
4	Command Center	Display live status, signals, timeline, recommendations, and approvals.
5	Agent chat and voice	Add structured agent responses and Tier 1 voice announcements.
6	Verification loop	Log outcomes and demonstrate recovery after approved action.
7	Polish and demo	Animations, empty states, mobile responsiveness, final presentation flow.

7. Construction Operating Model

PulseBuild models construction as a dependency network. A project contains tasks. Tasks require crews, materials, equipment, permits, inspections, and weather-compatible conditions. When one requirement changes state, a signal is created. That signal may affect one task or a chain of downstream tasks.

Entity	Meaning	Example
Project	Top-level job site being monitored.	Harbor Point Mixed-Use Build
Phase	Major work stage.	Sitework, Structure, MEP Rough-in, Finishes
Task	Schedulable work activity.	Pour slab, frame Level 2, install electrical rough-in
Crew	Trade group with availability.	Concrete Crew A, Electrical Crew B
Material	Required item or delivery.	Concrete, rebar, drywall, conduit
Equipment	Machine or tool needed for tasks.	Crane, lift, excavator, pump truck
Inspection	Required approval gate before dependent work.	Slab inspection before pour
Permit	External authorization needed to proceed.	Electrical permit, road closure permit
Signal	Event that changes operational state.	Concrete delivery delayed 4 hours
Recommendation	Agent-proposed response.	Move crew to interior prep and reschedule pour
Verification	Outcome check after action.	Crew reassigned and delivery confirmed

Operating Rule

A task is not only late when its planned end date passes. A task is at risk as soon as one required dependency becomes unavailable or uncertain. PulseBuild must detect this before the task visibly fails.

8. Reactive Agent Brain

The PulseBuild agent is an orchestrator over deterministic tools. It does not replace the business logic. It receives signal analysis, explains what matters, recommends a next action, and communicates in a professional construction-operations voice.

Stage	Owner	Required Output
1. Signal received	Simulation Engine or user input	Structured signal object with type, timestamp, project, related entity, and severity hint.
2. Classification	Signal Classifier	Signal type and normalized category.
3. Priority scoring	Priority Engine	Tier 1-4 with explanation and confidence.
4. Dependency resolution	Dependency Engine	Affected tasks, crews, inspections, materials, equipment, and milestones.
5. Impact prediction	Prediction Engine	Estimated delay, idle labor risk, cascade depth, and drift impact.
6. Recommendation	Recommendation Engine	One primary action, alternatives if applicable, expected benefit.
7. Approval check	Approval Engine	Autonomous, approval required, or blocked by safety boundary.
8. Verification	Verification Engine	Resolved, partially resolved, unresolved, or requires next action.
9. Explanation	AI Chat Layer	Situation, Priority, Evidence, Impact, Recommendation, Approval, Verification.
10. Audit	Timeline Logger	Permanent event record.

Agent Forbidden Behavior

- Do not make safety decisions autonomously.
- Do not invent real-time project data.
- Do not provide legal, contractual, or medical advice.
- Do not say a problem is resolved until verification confirms it.
- Do not hide uncertainty; if data is missing, say what is missing.
- Do not recommend unsafe shortcuts to maintain schedule.

9. Signal Taxonomy and Priority Matrix

Signals are the atomic inputs of PulseBuild. Every signal must be normalized into a category and priority tier before the rest of the reactive loop runs.

Signal Type	Default Tier	Trigger	Primary Reaction	Verification Check
Safety Incident	Tier 1	Reported hazard, injury, unsafe condition	Stop automation, alert human, require safety review	Human acknowledgement logged
Inspection Failed	Tier 1	Inspection status failed	Block dependent work, recommend recovery plan	Inspection rescheduled or passed
Inspection Missing	Tier 1	Required inspection absent before task start	Prevent task start, notify superintendent	Inspection scheduled or completed
Critical Equipment Failure	Tier 1	Required equipment unavailable for active task	Recommend backup equipment or resequence work	Equipment assigned or task moved
Material Delivery Delayed	Tier 2	Material not arriving before task need	Predict idle crew, recommend alternate task	New ETA or crew reassigned
Crew Shortage	Tier 2	Available workers below required workers	Recommend backup crew or reduced scope	Crew count restored or plan adjusted
Weather Alert	Tier 2	Weather conflicts with outdoor task	Move indoor work forward or reschedule outdoor work	Weather-compatible task active
Permit Pending	Tier 2	Permit not approved before dependent work	Block dependent task and notify coordinator	Permit approved or task resequenced
Task Behind Schedule	Tier 2	Actual progress below planned progress	Assess downstream impact and recommend recovery	Progress updated or recovery action approved
Minor Delay	Tier 3	Low-impact schedule slip	Monitor and show warning	No escalation needed
Task Complete	Tier 4	Task status completed	Update dependencies and health	Downstream tasks unlocked
Delivery Received	Tier 4	Material received	Update material status and active tasks	Material availability confirmed

Priority Rule

Priority may be upgraded when multiple low-risk signals combine. Example: rain alert + concrete pour + missing inspection becomes Tier 1 because productive work is blocked and downstream delay is likely.

10. Dependency, Drift, and Health Scoring

Health Score communicates operational stability. Drift Score communicates how far the project is moving away from plan. The MVP formulas should be deterministic and simple enough to explain in the UI.

Metric	MVP Formula / Rule	UI Behavior
Health Score	Start at 100. Tier 1 subtracts 20; Tier 2 subtracts 10; Tier 3 subtracts 5. Each affected downstream task subtracts 2. Unresolved recommendation subtracts 5. Verified recovery adds back up to 5.	Animated number. Green 85-100, amber 65-84, red below 65. Click opens calculation.
Drift Score	Tier 1 active x25 + Tier 2 active x15 + affected tasks x5 + idle crew risks x10 + unresolved actions x8.	Higher is worse. Show label Normal, Warning, High Drift, Critical Drift.
Cascade Depth	Number of downstream dependency hops affected by signal.	Show as dependency chain in Signal Detail.
Idle Labor Risk	Crew assigned to blocked task and no alternate active task available.	Show crew count and estimated hours at risk.
Recovery Confidence	High if action directly resolves dependency; medium if it mitigates; low if data missing.	Display confidence badge with explanation.

Dependency Logic

- A task is blocked if any required material, inspection, permit, equipment, or crew is unavailable.
- A downstream task is affected if it directly or indirectly depends on a blocked task.
- A crew is idle risk if assigned to a blocked task with no approved alternate work.
- A milestone is at risk if any task on its critical path is blocked or delayed.
- Cursor must implement these as pure functions before connecting the LLM.

11. Detailed Screen Specifications

Command Center / Home

The Command Center is the central screen. It must immediately show current project health, active blockers, recommendations, and live event history. A user should understand the highest operational risk without reading a long report.

Screen Area	Required UI Elements	Required User Actions
Top Bar	Project name, phase, date, simulation status, Start/Pause Simulation, settings icon	Start simulation, pause simulation, change demo speed later.
Health Strip	Health Score, Drift Score, Tier 1 count, unresolved actions, last update	Click score to view calculation breakdown.
Live Signal Feed	Newest signals, priority badges, source, related task, time	Open signal detail, filter by priority.
Recommendation Queue	Primary recommendation, approval status, expected impact, confidence	Approve, reject, request alternative, view evidence.
Agent Chat Panel	Chat history, suggested prompts, voice status	Ask blocker questions, approve actions, verify outcome.
Timeline	Chronological signal, analysis, approval, verification events	Review sequence and audit history.

Signal Detail Screen

- Show signal description, priority tier, evidence, related task, related entity, affected dependencies, predicted delay, idle crew risk, recommended action, approval requirement, verification plan, and audit status.
- Must include an explanation panel titled Why this matters.
- Must include a dependency chain view even if simple text-based for MVP.
- Must include status transitions: New, Analyzed, Recommendation Ready, Awaiting Approval, Executed, Verified, Closed.

12. Feature Specifications

Feature	Purpose	Core Rules	Acceptance Criteria
Live Signal Feed	Show operational events as they happen.	Newest first; priority badge required; every signal clickable; no hidden critical signals.	New simulated signal appears automatically and is logged.
Health Card	Summarize project stability.	Driven by deterministic score; cannot exceed 100 or fall below 0; explanation available.	Updates after each signal and verification.
Drift Card	Show deviation from plan.	Higher score means more operational drift; must list top causes.	User can identify why drift increased.
Recommendation Queue	Turn analysis into action.	One primary recommendation per Tier 1/2 signal; approval required for risky actions.	User can approve/reject and see outcome.
Approval Dialog	Prevent unsafe automation.	Must show action, reason, affected tasks, impact, and verification plan before approval.	No high-risk action executes without approval.
Verification Card	Close the loop.	Checks whether state changed after action; may propose next best action.	At least one full resolved and one unresolved state can be demoed.
Timeline	Permanent record.	Every signal/recommendation/approval/verification creates event.	Audit history is complete for demo scenario.
Chat Assistant	Conversational access to system.	Grounded in project data; structured response format; refuses out-of-scope requests.	User can ask What is blocking work? and receive correct answer.
Voice Alerts	Make critical events feel reactive.	Tier 1 only by default; no duplicate loops; user can mute.	Tier 1 signal triggers concise spoken alert.

13. Chat and Voice Agent Specification

The chat is not a general-purpose chatbot. It is the conversational interface to the Reflex Engine. It should allow the superintendent to ask operational questions without navigating multiple screens.

User Intent	Example Prompt	Agent Behavior
Status summary	What is blocking work right now?	Return top blocker, priority, evidence, affected tasks, and recommended action.
Reasoning request	Why did health drop?	Explain score changes using active signals and calculations.
Impact analysis	What happens if we ignore this?	Describe downstream tasks, idle labor, milestone risk, and uncertainty.
Approval	Approve the recommendation.	Confirm action, log approval, run execution simulation, start verification.
Rejection	Reject this action.	Log rejection and propose alternative if available.
Verification	Did the last action work?	Return verification status and next action if unresolved.

Required Response Format

Section	Required Content
Situation	One-sentence summary of event or question.
Priority	Tier and reason.
Evidence	Data points from source of truth only.
Impact	Affected tasks, crews, schedule, health/drift.
Recommendation	Concrete next best action.
Approval	Whether action requires user approval.
Verification	How the system will know whether the fix worked.

Voice Rules

- Use browser text-to-speech for MVP; do not build custom voice infrastructure.
- Announce Tier 1 events automatically unless muted.
- Keep voice messages under 20 seconds.
- Voice must sound calm and operational: "PulseBuild detected a Tier 1 blocker. Required inspection is missing before the concrete pour."
- Do not speak Tier 3 or Tier 4 events by default.

14. Command Center Live Interaction Model

PulseBuild must feel alive. The user should not need to refresh, ask, or search to know that a new blocker exists. The simulation engine drives events into the interface, and each event causes visible system reactions.

Event Step	UI Reaction	Agent Reaction
Signal arrives	Signal card slides into feed; timestamp appears; priority pending.	Analyzing signal.
Priority calculated	Badge changes to Tier 1-4; Health/Drift may animate.	Priority explanation prepared.
Dependencies found	Affected tasks highlighted; dependency chain appears.	Impact narrative generated.
Recommendation ready	Recommendation card appears in queue.	Chat posts concise action summary.
Approval required	Approval dialog or button activates.	Agent explains why approval is needed.
Action approved	Timeline logs approval; status becomes executing.	Agent confirms execution simulation.
Verification complete	Verification badge updates resolved/unresolved.	Agent explains outcome and next action if needed.

Microinteraction Rules

- New Tier 1 cards pulse for two seconds, then settle.
- Health Score transitions numerically instead of jumping abruptly.
- Recommendation card should feel urgent but not chaotic.
- Resolved events receive a completion state and move lower in the queue.
- No animation should block approval or make the interface feel slow.

15. Design System and Visual Identity

PulseBuild should feel futuristic, industrial, precise, and trustworthy. It should look like construction mission control, not consumer social software. The design should communicate speed, clarity, and operational authority.

Token	Hex	Usage
Command Navy	#0B1220	App shell, sidebar, header backgrounds
Steel Blue	#1E4E8C	Primary buttons, active states, charts
Signal Cyan	#27D3FF	Live activity, agent processing, highlights
Safety Red	#E5484D	Tier 1 blockers and safety-critical alerts only
Warning Amber	#F59E0B	Tier 2 high-risk signals
Build Green	#2FBF71	Resolved, verified, healthy states
Concrete Gray	#E5E7EB	Cards, borders, neutral surfaces
Slate Text	#111827	Primary text
Muted Text	#6B7280	Secondary labels

Detailed Design Rules

- Use dark/navy framing with light content cards or a premium dark-mode command center.
- Reserve red for true blockers and safety-related alerts.
- Use badges consistently for priority, status, confidence, and approval requirement.
- Use cards and timelines instead of dense tables in the user-facing app.
- Use monospaced or data-friendly typography for scores, times, and IDs.
- Use subtle glow or outline on live elements to communicate reactivity.
- Mobile must preserve the core loop: top blocker, signals, recommendation, chat.

16. Component Library

Component	Purpose	States / Variants
AppShell	Overall application frame	Desktop sidebar, tablet compact rail, mobile bottom nav.
CommandTopBar	Project context and simulation controls	Idle, running, paused, completed.
HealthCard	Operational health metric	Healthy, stable, at risk, critical.
DriftCard	Deviation from planned state	Normal, warning, high drift, critical drift.
SignalCard	Individual operational event	New, analyzing, analyzed, action ready, closed.
PriorityBadge	Signal severity	Tier 1, Tier 2, Tier 3, Tier 4.
TimelineItem	Audit event	Signal, analysis, recommendation, approval, verification.
RecommendationCard	Action proposal	Pending, approval required, approved, rejected, executing, verified.
ApprovalDialog	Human-in-the-loop confirmation	Default, warning, safety-locked.
VerificationCard	Resolution outcome	Resolved, partially resolved, unresolved, next action needed.
ChatPanel	Agent conversation	Collapsed, open, typing, voice active.
VoiceStatus	Voice announcement state	Muted, ready, speaking, disabled.
DependencyChain	Affected task chain	Collapsed, expanded, no downstream impact.
EmptyState	No active signals or no recommendations	No blockers, no history, simulation not started.

Component Rule

Cursor must reuse this component library. Do not create one-off cards for similar concepts. If a new component is necessary, Cursor must propose it before implementation.

17. Technical Architecture

Recommended MVP direction: React + Vite front end, local seeded JSON or lightweight local store for Phase 1, optional Supabase in later phase. The first build should prove the reactive experience before adding production infrastructure.

Layer	Technology / Pattern	Purpose
Front end	React + Vite	Modern component-based UI.
Routing	React Router if multi-screen	Command Center, Signal Detail, Settings, Logs.
State	Local store or Zustand if approved	Live simulation state, active signals, recommendations.
Data	Seeded JSON for MVP	Project, tasks, crews, materials, inspections, signals.
Domain logic	Pure TypeScript/JavaScript functions	Priority, dependencies, scoring, recommendations, verification.
Agent layer	LLM optional after domain logic	Explains deterministic outputs and handles chat.
Voice	Browser SpeechSynthesis API	Tier 1 voice alerts without external dependency.
Animation	CSS transitions or Framer Motion if approved	Live feel, card entrance, score transitions.
Testing	Unit tests for domain logic	Ensure deterministic scoring and decisions.

Approved Repository Direction

- Repository name: pulsebuild-app.
- Documentation belongs in docs/.
- Seeded data belongs in src/data/ or data/.
- Domain logic belongs in src/domain/ and must be testable without UI.
- Agent orchestration belongs in src/agent/ and must call domain logic rather than duplicate it.
- UI components belong in src/components/.
- Simulation engine belongs in src/simulation/.
- Never commit secrets. Use .env.example for future API placeholders.

18. Conceptual Database Schema

This schema is conceptual and should first be implemented as seeded JSON. Cursor must not create production migrations until approved.

Entity	Purpose	Core Fields
projects	Construction project being simulated	id, name, location, phase, start_date, target_date, health_score, drift_score, status
phases	Major construction stages	id, project_id, name, planned_start, planned_end, status
tasks	Schedulable work items	id, project_id, phase_id, name, status, planned_start, planned_end, actual_start, actual_end, dependencies, required_crew_ids, required_material_ids, required_equipment_ids, required_inspection_ids, priority_weight
crews	Labor teams	id, name, trade, required_workers, available_workers, current_task_id, status
materials	Required supplies	id, name, delivery_status, expected_delivery_time, quantity_expected, quantity_received, related_task_ids
equipment	Machines/tools	id, name, status, assigned_task_id, available_from, required_for_task_ids
inspections	Approval gates	id, type, status, required_before_task_id, scheduled_time, inspector_name
permits	External approvals	id, type, status, required_for_task_id, expiration_date
weather_events	Weather risks	id, timestamp, condition, severity, affected_task_types, start_time, end_time
signals	Operational events	id, project_id, timestamp, signal_type, description, source, related_entity_type, related_entity_id, severity_hint, status
recommendations	Agent actions	id, signal_id, action_type, recommendation, requires_approval, confidence, expected_impact, status
reactions	User decisions/executions	id, recommendation_id, approved_by, approval_status, executed_at, notes
verifications	Outcome checks	id, reaction_id, verification_status, verified_at, result_summary, next_best_action

19. Simulation Engine

The simulation engine is essential because the MVP does not integrate with live construction systems. It creates the experience of a real-time reactive job site by injecting realistic events into the system at controlled intervals.

Simulation Requirement	Specification
Start button	Begins event sequence and changes status to Running.
Pause button	Stops new events but preserves current state.
Event interval	Default every 5-10 seconds; demo speed may be adjustable.
Scenario support	At least one scripted scenario: Concrete pour disrupted by missing inspection, rain, and delivery delay.
State mutation	Events update related entities, not only the signal feed. Example: material delay updates material.delivery_status.
Determinism	Same scenario should produce same result unless Product Owner enables randomness.
Demo reset	Reset project state to initial seed.
Verification triggers	After approved action, simulation should update related state so verification can succeed or fail.

Required Demo Scenario

- Project starts healthy with Health Score above 90.
- Rain alert enters system and creates Tier 2 weather risk.
- Concrete delivery delay enters system and affects slab pour.
- Inspection missing enters system and upgrades to Tier 1 blocker.
- Agent recommends delaying pour and reassigning crew to indoor prep.
- User approves recommendation.
- System verifies crew reassigned and blocker mitigated.
- Health Score partially recovers and timeline logs full loop.

20. Cursor Rules and Development Workflow

Cursor must treat this document as the authoritative working reference. The goal is a polished MVP, not uncontrolled feature expansion.

Rule	Instruction
Read before coding	Cursor must read this Source of Truth before scaffolding or modifying major files.
Do not invent scope	If a feature is not described here, ask before adding it.
Domain first	Implement signal classification, scoring, dependency analysis, and verification as pure functions before UI.
No LLM dependency first	The app must work with deterministic logic even if no LLM is connected.
Agent grounding	Agent output must be based on data returned by domain logic.
Component reuse	Use approved components from the component library.
Approval boundaries	Never bypass human approval for risky operational changes.
No secrets	Do not commit API keys or credentials.
Explain dependencies	Before installing packages, explain need, alternatives, and risk.
Phase approval	After each milestone, summarize completed work and ask for Product Owner approval before continuing.

Implementation Order

- 1. Create docs folder and store this PDF plus markdown mirror.
- 2. Create seed data files.
- 3. Build domain logic tests.
- 4. Build Command Center UI with static data.
- 5. Connect simulation engine.
- 6. Add recommendation and approval flow.
- 7. Add verification loop.
- 8. Add chat and voice layer.
- 9. Polish visual system and responsive layout.

21. Acceptance Criteria, Testing, and Definition of Done

Area	Acceptance Criteria
Signal ingestion	New simulated signals appear in live feed, receive priority tier, and create timeline entries.
Priority engine	All seeded demo signals classify into expected tiers.
Dependency engine	Affected tasks and downstream chains appear for material, inspection, and crew signals.
Health/Drift	Scores update after signal processing and after verification.
Recommendation	Every Tier 1 and Tier 2 signal receives a clear recommended action.
Approval	Actions requiring approval cannot execute until user approves.
Verification	At least one action resolves and one action can remain unresolved with next-best recommendation.
Chat	User can ask top blocker, health explanation, impact, recommendation, and verification questions.
Voice	Tier 1 event triggers optional spoken alert and respects mute state.
Timeline	Every major state change is logged with timestamp and status.
UI quality	Command Center looks polished, responsive, and interactive.
Cursor compliance	Code follows folder structure and deterministic domain-first architecture.

Definition of Done

- User starts simulation from Command Center.
- Signals enter automatically and visibly change UI state.
- System identifies highest priority blocker.
- Agent explains situation, priority, evidence, impact, recommendation, approval, and verification.
- User approves a corrective action.
- System updates timeline, recalculates scores, and verifies the outcome.
- Final demo clearly shows PulseBuild is reactive, not just a dashboard.

22. Decision Log Seed

Decision	Approved Direction	Reason
Product category	Reactive construction operations agent	Differentiates from static project management dashboards.
MVP data	Seeded simulation	Avoids integration complexity while proving concept.
Agent role	Orchestrator and explainer over deterministic logic	Prevents hallucination and improves reliability.
Interface model	Command Center plus chat	Supports live monitoring and conversational explanation.
Voice	Browser text-to-speech for Tier 1 only	Adds wow factor without building voice infrastructure.
Safety boundary	Always human-approved	Construction safety cannot be automated in MVP.
Scoring	Simple deterministic formulas	Explainable and easy to implement.
Architecture	React first, optional Supabase later	Prioritizes demo speed and interactive quality.

Open Questions for Product Owner

- Should the first MVP be dark-mode mission control or light-mode professional SaaS?
- Should chat be always open or collapsible by default?
- Should simulation events be scripted only or allow random event mode?
- Should the first build include Supabase or remain local until UI and domain logic are approved?
- Which project scenario should be used for the final class demo?

23. Cursor Phase 1 Completion Prompt

Use this prompt after uploading the PDF to Cursor.

You are building PulseBuild, a reactive construction operations agent. Read the Source of Truth v2.0 PDF before coding. Do not invent product scope. First, create a clean project plan and folder structure only. Do not implement features yet. Your plan must include: data files, domain logic modules, simulation engine, command center UI components, agent/chat layer, verification loop, and testing plan. Respect the rule that business logic must be deterministic first and the LLM must never invent project facts. After planning, stop and ask for Product Owner approval.

Cursor must confirm the following before implementation:

- I understand PulseBuild is a reactive construction operations agent, not a generic chatbot.
- I will build deterministic domain logic before the agent layer.
- I will use seeded MVP data unless instructed otherwise.
- I will not bypass human approval boundaries.
- I will follow the approved component library and architecture.
- I will stop after planning and request approval.